ECOLE NATIONALE DES SCIENCES APPLIQUEES

Beni Mellal

Pr. BE ELBAGHAZAOUI

COMPTE RENDU

TP N°4

Architecture Orientee Services (SOA)

Application E-Commerce avec Spring Boot & Microservices

Etudiant :	Achraf HAMOUDA
Module :	Architecture Logicielle
Date :	30 Mars 2026
Encadrant :	Pr. BE ELBAGHAZAOUI

Annee Universitaire 2025–2026

Contents

1 Introduction **2**
1.1 Objectifs pedagogiques 2
1.2 Prerequis utilises 2

2 Architecture du Systeme **2**

3 Structure du Projet **3**

4 Implementation de base **3**
4.1 Eureka Server (port 8761) 3
4.2 Product Service (port 8081) 3
4.3 Order Service (port 8082) 4

5 Tests de base avec Postman **4**
5.1 Creer un produit 4
5.2 Lister les produits 5
5.3 Creer une commande 5
5.4 Lister les commandes 6

6 Exercices pratiques **6**
6.1 Exercice 1 – Mise a jour du stock 6
6.2 Exercice 2 – API Gateway 8
6.3 Exercice 3 – Gestion des erreurs 9

7 Recapitulatif des tests **10**

8 Conclusion **10**

1. Introduction

Ce travail pratique porte sur la mise en oeuvre d'une **Architecture Orientee Services (SOA)** a l'aide du framework **Spring Boot**. L'objectif est de concevoir et deployer une application e-commerce composee de plusieurs microservices independants qui communiquent entre eux via des APIs REST.

1.1 Objectifs pedagogiques

- Comprendre les principes fondamentaux de l'architecture SOA
- Creer et configurer des microservices avec Spring Boot 3.2.x
- Implementer la communication inter-services via REST et Feign Client
- Gerer une base de donnees embarquee avec Spring Data JPA (H2)
- Utiliser un service de decouverte (Eureka Server)
- Implementer une API Gateway avec Spring Cloud Gateway
- Gerer les erreurs inter-services de facon robuste

1.2 Prerequis utilises

- Java JDK 17.0.12 (Oracle) — Apache Maven 3.9.14
- Claude Code (generation et demarrage automatique des services)
- Postman (tests des APIs REST)
- IntelliJ IDEA (visualisation du code)

2. Architecture du Systeme

L'application finale est composee de **quatre microservices**, chacun independant avec sa propre responsabilite.

Service	Port	Role	Base de donnees
`eureka-server`	8761	Annuaire / Decouverte	–
`product-service`	8081	Catalogue + gestion stock	H2 : productdb
`order-service`	8082	Commandes + gestion erreurs	H2 : orderdb
`api-gateway`	8080	Point d'entree unique	–

3. Structure du Projet

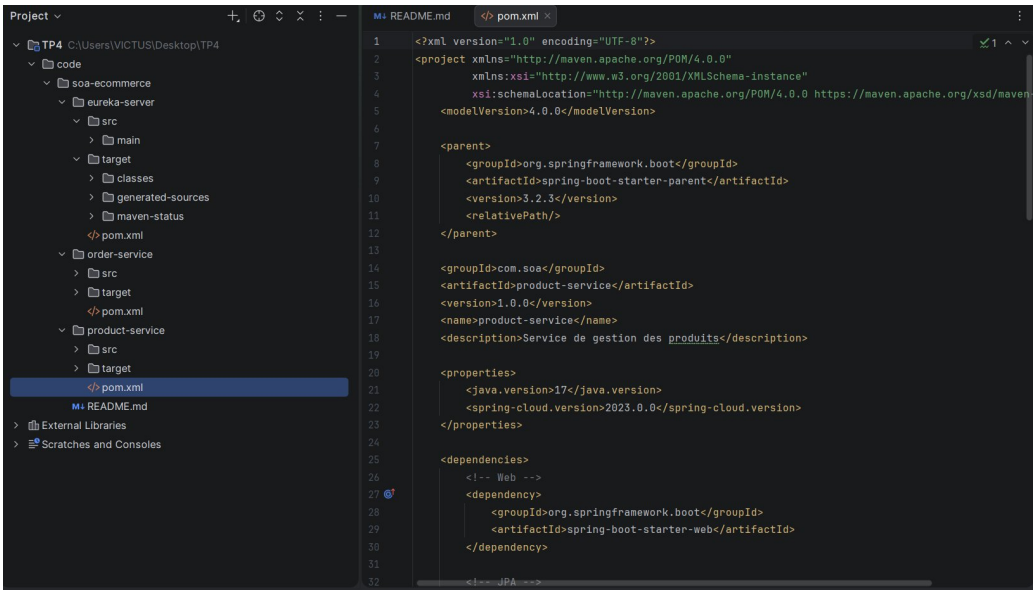

Figure 1. Structure du projet dans IntelliJ IDEA – les 3 microservices principaux

4. Implementation de base

4.1 Eureka Server (port 8761)

Listing 1. application.properties – Eureka Server

```
server.port=8761
spring.application.name=eureka-server
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
```

Listing 2. EurekaServerApplication.java

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

4.2 Product Service (port 8081)

Endpoints exposes : `GET /api/products`, `GET /api/products/{id}`, `POST /api/products`, `DELETE /api/products/{id}`

Listing 3. Entite Product.java

```
@Entity @Data @NoArgsConstructor @AllArgsConstructor
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
```

```
    private String description;
    private Double price;
    private Integer stock;
}
```

4.3 Order Service (port 8082)

Listing 4. Feign Client – ProductClient.java

```
@FeignClient(name = "product-service")
public interface ProductClient {
    @GetMapping("/api/products/{id}")
    ProductDTO getProductById(@PathVariable Long id);
}
```

Listing 5. OrderService.java – createOrder()

```
public Order createOrder(Long productId, Integer quantity) {
    ProductDTO product = productClient.getProductById(productId);
    Order order = new Order();
    order.setProductId(productId);
    order.setProductName(product.getName());
    order.setQuantity(quantity);
    order.setTotalPrice(product.getPrice() * quantity);
    order.setOrderDate(LocalDateTime.now());
    order.setStatus("PENDING");
    return orderRepository.save(order);
}
```

5. Tests de base avec Postman

5.1 Creer un produit

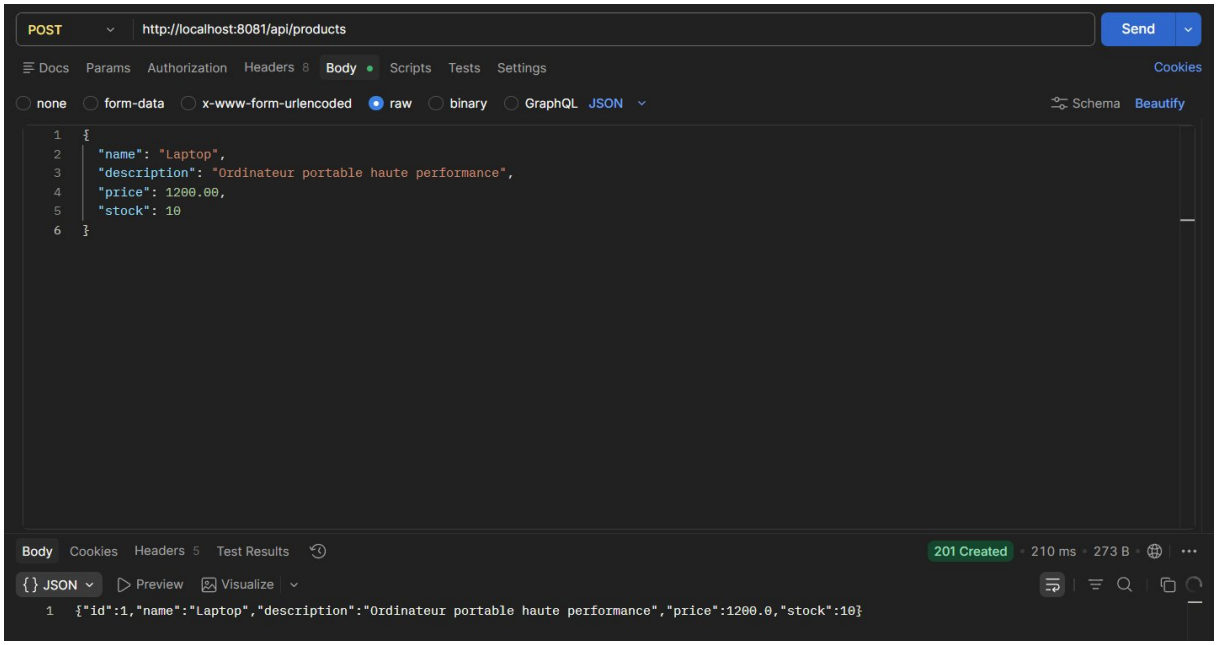

Figure 2. POST /api/products – 201 Created, produit id=1 cree

Resultat

`HTTP 201 Created` – Produit Laptop cree avec id=1, price=1200.0, stock=10.

5.2 Lister les produits

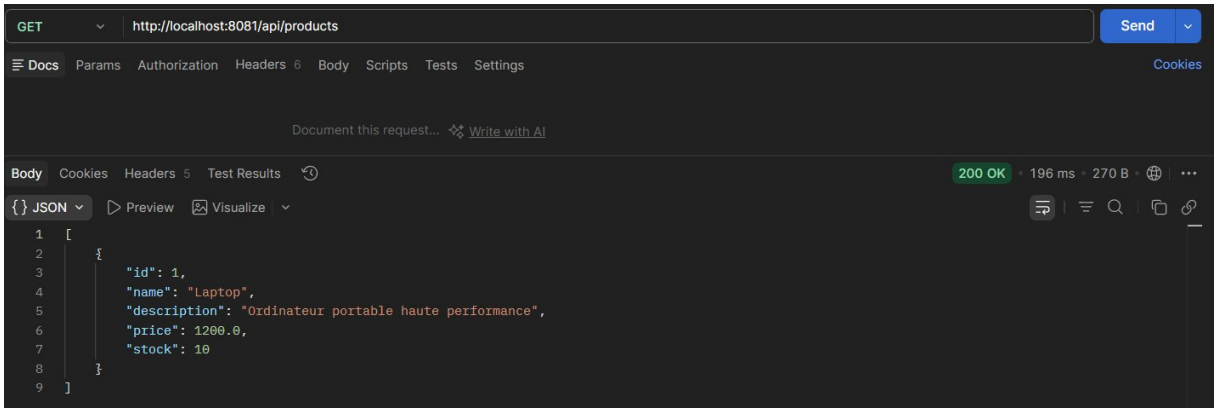

Figure 3. GET /api/products – 200 OK, liste des produits

5.3 Creer une commande

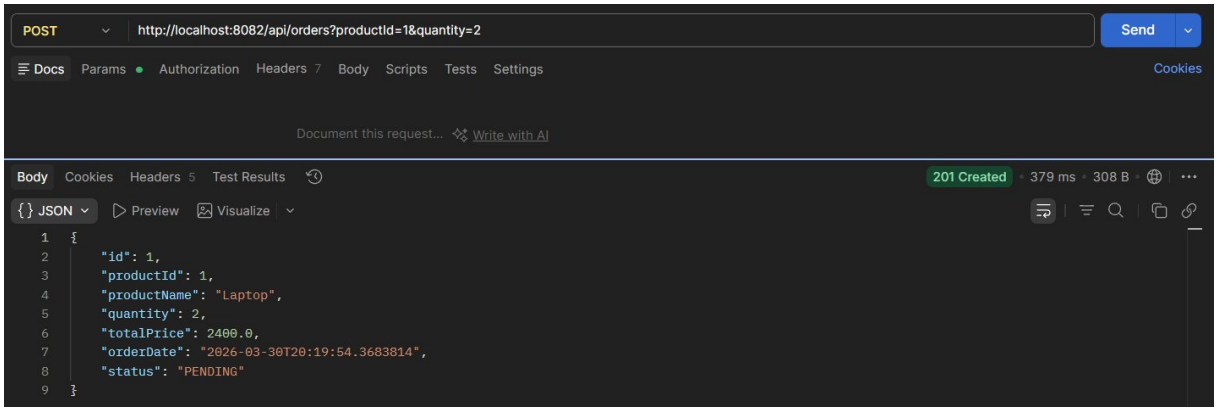

Figure 4. POST /api/orders?productId=1&quantity=2 – totalPrice calcule automatiquement

Resultat

`HTTP 201 Created` – totalPrice = 1200.0 x 2 = **2400.0**, status=PENDING. Le Feign Client a bien interroge le Product Service.

5.4 Lister les commandes

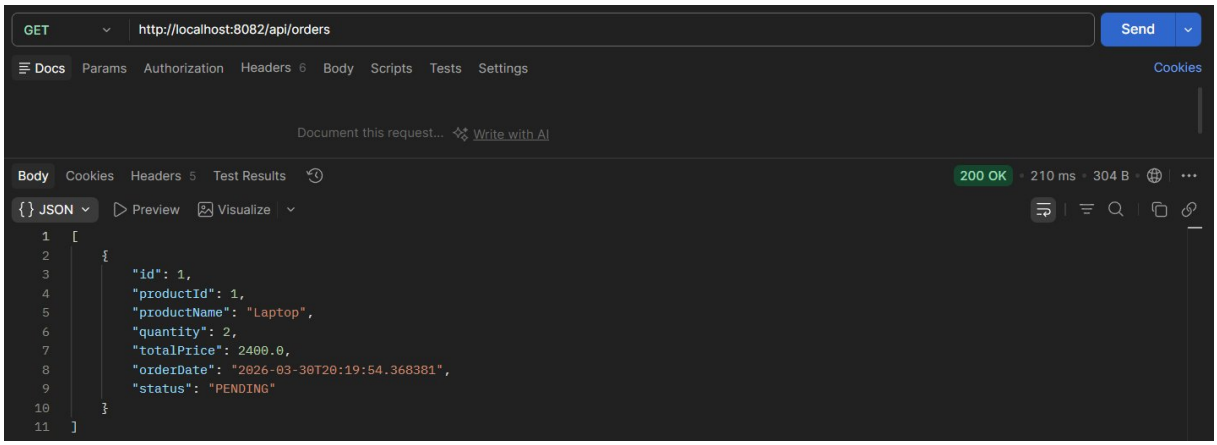

Figure 5. GET /api/orders – 200 OK, commande persistee

6. Exercices pratiques

6.1 Exercice 1 – Mise a jour du stock

Objectif : ajouter un endpoint `PUT /api/products/{id}/stock` qui reduit le stock apres une commande, et gerer le cas ou le stock est insuffisant.

Modifications apportees :

- Nouveau endpoint `PUT /api/products/{id}/stock?quantity=X` dans `ProductController`
- Validation : retourne `400 Bad Request` si stock $<$ quantity
- `OrderService` appelle `updateStock()` apres sauvegarde de la commande
- `ProductClient` mis a jour avec la nouvelle signature

Listing 6. ProductService.java – updateStock()

```java
public Product updateStock(Long id, int quantity) {
    Product product = productRepository.findById(id)
        .orElseThrow(() -> new RuntimeException("Produit introuvable"
            ));
    if (product.getStock() < quantity) {
        throw new IllegalArgumentException("Stock insuffisant");
    }
    product.setStock(product.getStock() - quantity);
    return productRepository.save(product);
}
```

Test 1 – Stock mis a jour (200 OK) :

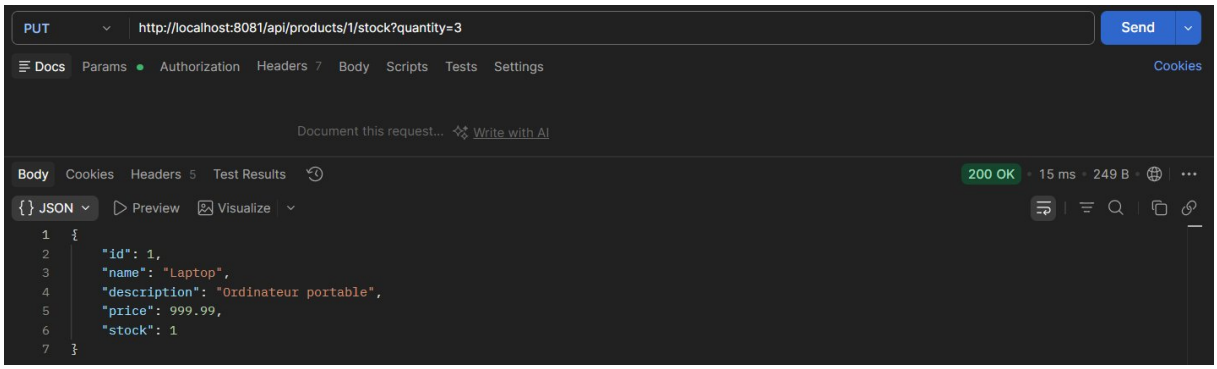

Figure 6. PUT /api/products/1/stock?quantity=3 – stock reduit avec succes

Resultat

`HTTP 200 OK` – Le stock est bien reduit apres l'appel. Le produit retourne le nouveau stock.

Test 2 – Stock insuffisant (400 Bad Request) :

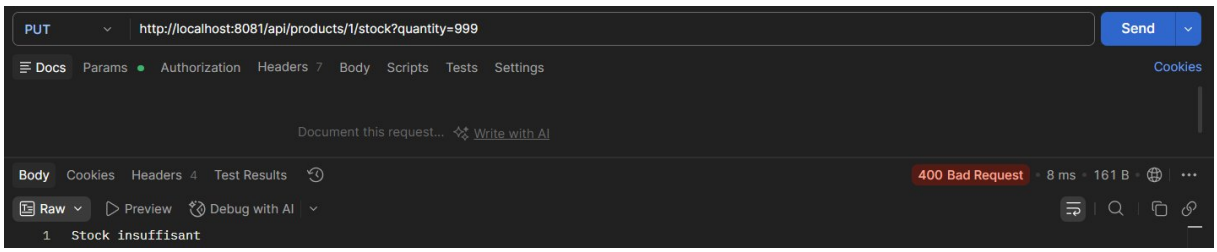

Figure 7. PUT /api/products/1/stock?quantity=999 – 400 "Stock insuffisant"

Erreur geree

`HTTP 400 Bad Request` – Message : "Stock insuffisant". La commande n'est pas creee.

Test 3 – Verification du stock apres commande :

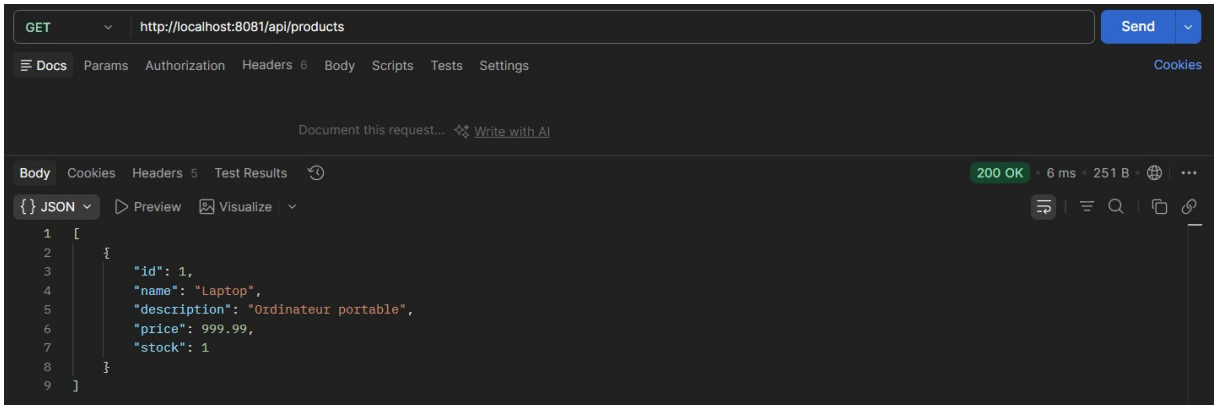

Figure 8. GET /api/products – stock reduit automatiquement apres la commande

Resultat

`HTTP 200 OK` – Le stock du Laptop est bien reduit apres chaque commande passee.

6.2 Exercice 2 – API Gateway

Objectif : creer un service `api-gateway` sur le port 8080 qui centralise toutes les requetes et les redirige vers les bons microservices.

Configuration de l'API Gateway :

Listing 7. application.yml – api-gateway

```
server.port=8080
spring.application.name=api-gateway
spring.cloud.gateway.routes[0].id=product-service
spring.cloud.gateway.routes[0].uri=lb://product-service
spring.cloud.gateway.routes[0].predicates[0]=Path=/products/**
spring.cloud.gateway.routes[1].id=order-service
spring.cloud.gateway.routes[1].uri=lb://order-service
spring.cloud.gateway.routes[1].predicates[0]=Path=/orders/**
```

Dashboard Eureka avec 4 services enregistres :

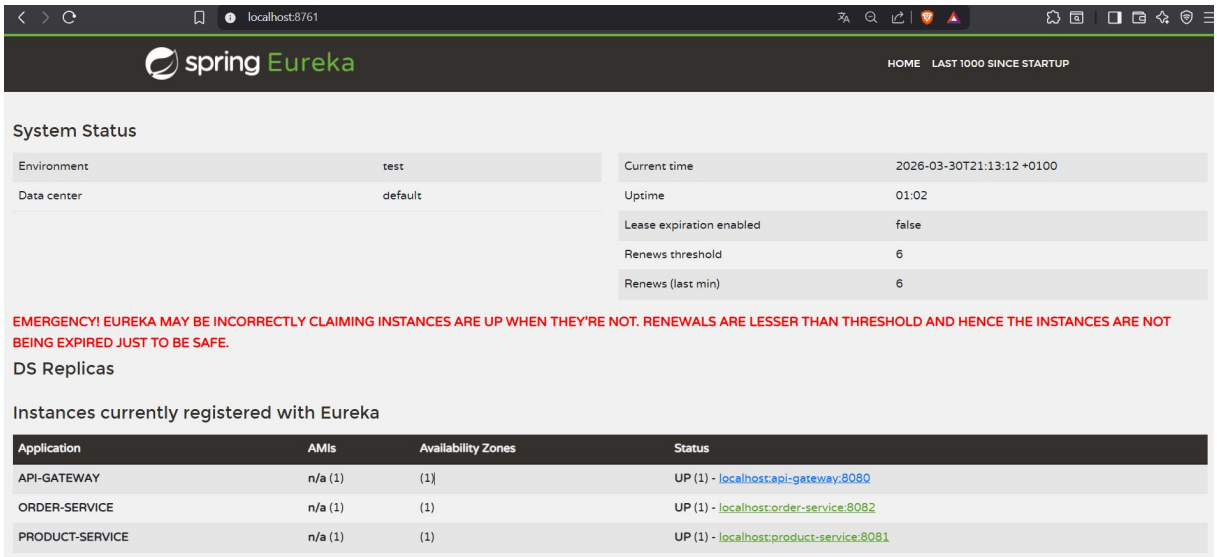

Figure 9. Eureka Dashboard – API-GATEWAY, ORDER-SERVICE et PRODUCT-SERVICE tous UP

Test – GET products via Gateway (port 8080) :

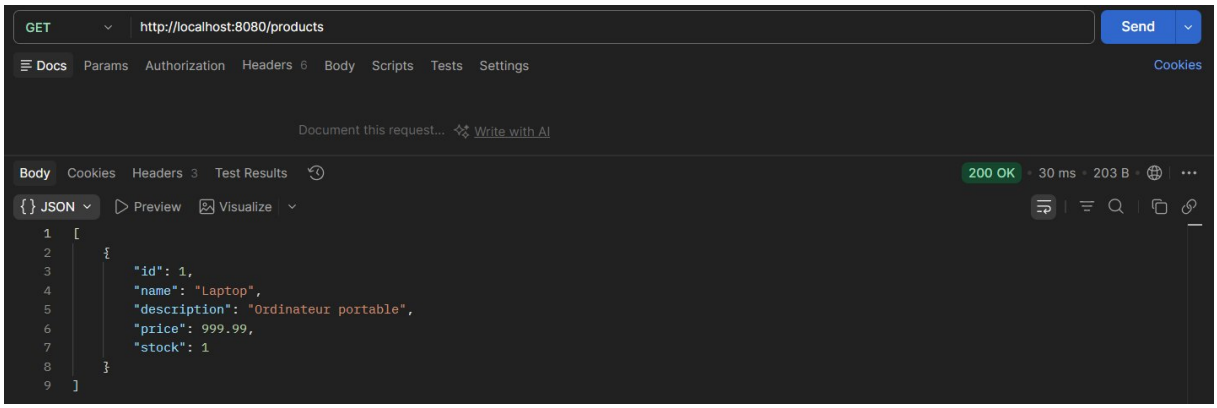

Figure 10. GET http://localhost:8080/products – redirige vers product-service, 200 OK

Resultat

`HTTP 200 OK` – La requete sur le port 8080 est bien redirigee vers `product-service:8081`.

Test – GET orders via Gateway (port 8080) :

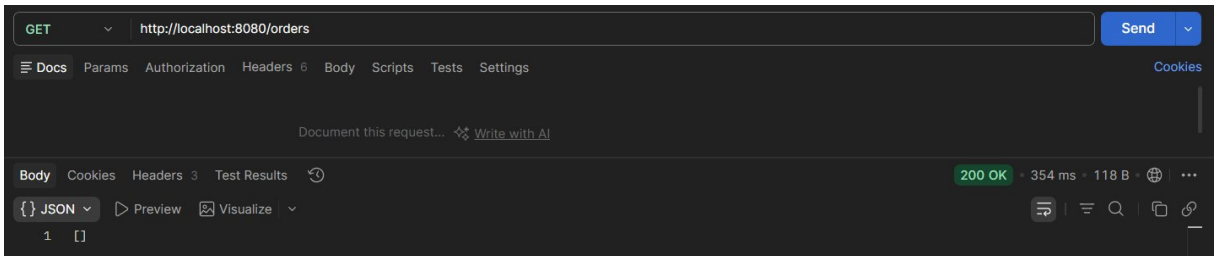

Figure 11. GET http://localhost:8080/orders – redirige vers order-service, 200 OK

Resultat

`HTTP 200 OK` – La requete sur le port 8080 est bien redirigee vers `order-service:8082`.

6.3 Exercice 3 – Gestion des erreurs

Objectif : ameliorer la robustesse du `order-service` en gerant les erreurs liees aux appels Feign (produit inexistant, service indisponible).

Classes creees :

- `ErrorResponse.java` : structure de reponse d'erreur (timestamp, status, message)
- `GlobalExceptionHandler.java` : gestionnaire global avec `@RestControllerAdvice`

Listing 8. GlobalExceptionHandler.java

```java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(FeignException.NotFound.class)
    public ResponseEntity<ErrorResponse> handleNotFound(
        FeignException.NotFound ex) {
        return ResponseEntity.status(404)
            .body(new ErrorResponse(LocalDateTime.now(), 404, "
                Produit introuvable"));
    }

    @ExceptionHandler(FeignException.class)
    public ResponseEntity<ErrorResponse> handleFeign(FeignException
        ex) {
        return ResponseEntity.status(503)
            .body(new ErrorResponse(LocalDateTime.now(), 503,
                "Service Product indisponible"));
    }

    @ExceptionHandler(IllegalArgumentException.class)
    public ResponseEntity<ErrorResponse> handleIllegal(
        IllegalArgumentException ex) {
```

```
        return ResponseEntity.status(400)
            .body(new ErrorResponse(LocalDateTime.now(), 400, ex.
                getMessage()));
    }
}
```

Test – Produit inexistant (404) :

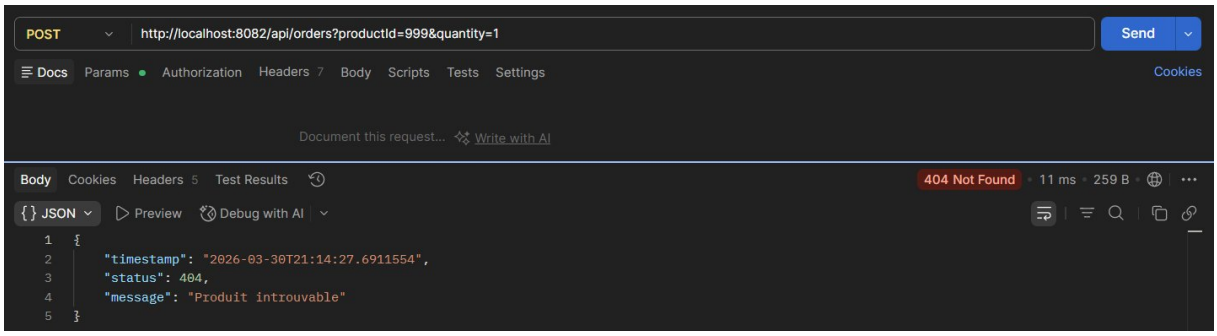

Figure 12. POST /api/orders?productId=999&quantity=1 – 404 "Produit introuvable"

Erreur geree

`HTTP 404 Not Found` – Reponse structuree avec timestamp, status=404 et message "Produit introuvable". L'`ErrorResponse` est bien retourne.

7. Recapitulatif des tests

Test	Methode	URL	Code	OK
Creer produit	POST	`:8081/api/products`	201	+
Lister produits	GET	`:8081/api/products`	200	+
Creer commande	POST	`:8082/api/orders`	201	+
Lister commandes	GET	`:8082/api/orders`	200	+
Reduire stock	PUT	`:8081/.../stock?quanti`	200	+
Stock insuffisant	PUT	`:8081/.../stock?quantity=999`	400	+
Via Gateway products	GET	`:8080/products`	200	+
Via Gateway orders	GET	`:8080/orders`	200	+
Produit inexistant	POST	`:8082/api/orders?id=99`	404	+

8. Conclusion

Ce TP a permis de mettre en pratique les concepts fondamentaux de l'**Architecture Orientee Services** avec Spring Boot. Les quatre microservices deployes (Eureka, Product, Order, API Gateway) illustrent les avantages de la SOA : modularite, independance et reutilisabilite.

Les trois exercices ont permis d'aller plus loin :

- **Exercice 1** : la gestion du stock garantit la coherence des donnees entre les services.

- **Exercice 2** : l'API Gateway centralise les acces et simplifie la configuration cote client.
- **Exercice 3** : la gestion des erreurs rend l'application robuste face aux pannes inter-services.

Cette architecture est directement applicable en production avec des adaptations (bases de donnees persistantes, securite OAuth2, monitoring avec Prometheus/Grafana).